

Reproduction and Architectural Ablation of PPO and DQN for Discrete-Action Autonomous Highway Driving

SHAO-MING, RUAN

Institute of Electrical Engineering (Group F)

National Cheng Kung University

Tainan, Taiwan

Abstract—Decision-making for autonomous driving is naturally framed as a sequential decision problem, making it a strong testbed for deep reinforcement learning (RL). This project *reproduces and ablates* two canonical deep-RL algorithms—Proximal Policy Optimization (PPO) and Double Deep Q-Network (Double DQN)—on the discrete-action `highway-env` driving simulator. The goal is to investigate two practical design choices: (i) does a permutation-aware Self-Attention feature extractor outperform a plain Multi-Layer Perceptron (MLP) under a strict parameter-matched budget, and (ii) how do on-policy and off-policy methods compare in both sample efficiency and resulting driving behaviour?

My results show that under the aggressive reward style ($n = 5$ seeds), PPO with Self-Attention (235.7 ± 16.7), PPO with MLP (223.0 ± 19.6), and Double DQN (234.1 ± 7.7) all approach a comparable theoretical reward ceiling (≈ 266.7). However, Double DQN achieves the lowest cross-seed variance ($\sigma = 7.7$) and faster initial learning. Critically, behavioural evaluation reveals a stark contrast: Double DQN achieves a significantly lower crash rate (11.3%) compared to the extreme bimodal behaviours observed in PPO variants (60.7–80.0% crash rates). Because these metrics are measured from deterministic policies and my reward styles differ only in reward coefficients, this gap reflects a genuine difference in the *learned policies*, not a test artifact—demonstrating that episodic return alone is an insufficient metric for safety-critical environments. This pattern *generalises* to a second environment (`merge-v0`), where the three methods tie even more tightly on return ($\sigma \leq 0.3$) yet PPO crashes in 100% of episodes against DQN’s 0%. A reward-shaping study further confirms that qualitative driving style is overwhelmingly determined by reward design rather than architectural choice, yet convergence to safe behavioural basins remains highly sensitive to optimisation variance. All code and trained models are released for full reproducibility.

Index Terms—Reinforcement learning, Proximal Policy Optimization, Deep Q-Network, self-attention, autonomous driving, ablation study, `highway-env`.

I. INTRODUCTION

Autonomous-driving decision-making—deciding *when* to change lanes, accelerate, or yield—can be modelled as a Markov Decision Process (MDP) in which an agent repeatedly observes nearby traffic and selects a high-level manoeuvre. Deep reinforcement learning (RL) is an attractive solution because it learns a closed-loop policy directly from interaction, without hand-engineering the trade-off between progress and safety. The `highway-env` simulator has become a standard, lightweight benchmark for exactly this class of problems: it

exposes a small kinematic state, a discrete set of meta-actions, and a configurable reward that trades speed against collisions.

A practitioner approaching this benchmark faces two recurring design choices that are rarely studied in a controlled way:

- 1) **Feature extractor.** The observation is a *set* of vehicles (the main vehicle and its N nearest neighbours). A natural inductive bias is to process this set with **self-attention**, which is permutation-equivariant and lets the main vehicle attend to the most relevant neighbours. The simpler baseline is to **flatten** the set and feed a plain MLP. Does the attention bias actually help when the two networks are given the *same* parameter budget and the *same* training pipeline?
- 2) **Algorithm family.** PPO (on-policy) and DQN (off-policy) are the two workhorses of deep RL. Their relative sample efficiency and final behaviour on this benchmark are folklore; I measure them under identical conditions.

Positioning. Applying self-attention to an ego-plus- N -vehicle observation is a *standard* design in `highway-env`; reproducing it is not my claim to novelty. My contribution is an **empirical finding with methodological teeth**: under a controlled, parameter-fair comparison I show that *episodic return—the metric almost universally reported on this benchmark—can be actively misleading for safety-critical control*, and I quantify by how much.

Contributions.

- **The central finding.** Across a parameter-fair ablation, all three configurations land within 5.7% on episodic return, yet their deployed crash rates differ by an order of magnitude (11.3% for DQN vs. 60–80% for PPO). I replicate this on a *second* environment (`merge-v0`), where returns tie within 1.4% but PPO crashes in 100% of episodes against DQN’s 0%. I argue this makes **behaviour-level evaluation a necessary complement to return curves** on this benchmark, not an optional extra.
- **A parameter-matched architectural ablation**—Self-Attention vs. MLP actor-critic (66.8k vs. 72.3k parameters, a 7.5% gap)—isolating the inductive bias from model capacity, alongside an off-policy Double-DQN baseline.

- A from-scratch, readable implementation of **PPO** (GAE, clipped surrogate, per-minibatch advantage normalisation, linear LR decay, gradient clipping) and **Double DQN** (replay buffer, ϵ -greedy, target network, double-Q target), cross-checked against **Stable-Baselines3** (Section IV).
- **Multi-seed** training ($n = 5$) with **mean $\pm$ std** learning curves, and a **reward-shaping study** (conservative / base / aggressive) showing reward design—not architecture—dominates qualitative driving style.

How this study came about. I did not set out to study evaluation metrics. I started by reproducing the standard attention baseline as a warm-up, expecting the architecture comparison to be the story. What redirected the project was an accident of observation: the three configurations reached almost the same episodic return, but when I actually watched them drive they behaved nothing alike—one cruised, another drove straight into traffic. That mismatch between “the number” and “the behaviour” is what this paper ended up being about.

All claims are scoped to the experimental conditions studied.

II. BACKGROUND AND RELATED WORK

Proximal Policy Optimization (PPO) [1] is an on-policy policy-gradient method that maximises a *clipped surrogate objective* to keep each update close to the data-collecting policy, giving much of the stability of trust-region methods at first-order cost. It is combined with **Generalized Advantage Estimation (GAE)** [2], which interpolates between high-bias/low-variance TD and low-bias/high-variance Monte-Carlo advantage estimates via parameter λ .

Deep Q-Networks (DQN) [3] learn an off-policy value function with experience replay and a target network. **Double DQN** [4] decouples action *selection* from action *evaluation* in the bootstrap target to reduce the well-known over-estimation bias of vanilla Q-learning.

Attention for driving. Leurent and Mercat [5] proposed a *social attention* architecture for tactical decision-making in dense traffic, letting the ego vehicle attend over a variable number of neighbours. This design is now a common baseline shipped with `highway-env`. My use of it is a *reproduction*, used here as one arm of a controlled ablation.

Benchmark. `highway-env` [6] is a collection of tactical-driving tasks (highway, merge, roundabout, intersection) with configurable observation, action, dynamics, and reward. I use the `Kinematics` observation and the `DiscreteMetaAction` action space throughout.

III. PROBLEM FORMULATION

I model each task as an episodic MDP (S, A, P, r, γ) .

State / observation. The `Kinematics` observation is a matrix of shape $V \times F$ with $V = 5$ vehicles (the ego vehicle plus its four nearest neighbours) and $F = 5$ features per vehicle: presence, x , y , v_x , v_y . Coordinates and velocities are ego-relative (`absolute=False`) and normalised to roughly $[-1, 1]$ (`normalize=True`). The observation is

TABLE I
REWARD COEFFICIENTS PER DRIVING STYLE (`HIGHWAY-V0`).

Style	coll.	hi-sp	lc	rt-lane	spd (m/s)
base	-1.0	0.4	0.0	def.	[20, 30]
conservative	-2.0	0.4	-0.5	def.	[10, 20]
aggressive	-4.0	2.0	+0.2	0.0	[30, 40]

therefore a small *set* of vehicles, motivating the permutation-aware attention model.

Action. `DiscreteMetaAction` exposes five high-level manoeuvres: 0 = `LANE_LEFT`, 1 = `IDLE`, 2 = `LANE_RIGHT`, 3 = `FASTER`, 4 = `SLOWER`. The agent acts at `policy_frequency = 5` Hz while physics runs at `simulation_frequency = 15` Hz.

Episode horizon (a subtle point I learned the hard way). I initially assumed `duration = 80` meant 80 decision steps, and my early reward numbers refused to add up—episodes seemed to end “too late” for the per-step rewards I was logging. Tracing `self.time` in the source resolved it: `highway-env`’s duration is measured in *simulated seconds*, not steps. Truncation fires when `self.time  $\geq$  duration`, and `self.time` advances by `1/policy_frequency = 0.2` s per decision. Therefore `duration = 80` yields a **maximum of $80 \times 5 = 400$ decision steps per episode**, not 80 steps. Training executes 100,000 environment steps ≈ 97 PPO updates.

Reward shaping. `highway-env`’s reward trades high speed against collisions and lane-change penalties. I expose three *driving styles* that change the reward coefficients (Table I). These coefficients were not chosen analytically but tuned by hand until the behaviour matched the label: for the aggressive style I first kept the default collision penalty of -1.0 , but with `high_speed_reward = 2.0` the agent simply ignored collisions and floored it; only after raising the penalty to -4.0 did crashing start to register in its decisions. Crucially, the **absolute return scale differs between styles** (e.g. aggressive multiplies the high-speed term), so episodic returns are only comparable *within* a style. Cross-style comparison is performed with behaviour metrics instead.

IV. METHODS

A. PPO (from scratch)

At each iteration the agent collects a rollout of $T = 1024$ transitions, then performs $K = 10$ epochs of minibatch SGD. Advantages are computed with GAE:

$$\delta_t = r_t + \gamma V(s_{t+1}) (1 - d_{t+1}) - V(s_t), \quad (1)$$

$$A_t = \delta_t + \gamma \lambda (1 - d_{t+1}) A_{t+1}, \quad (2)$$

$$R_t = A_t + V(s_t) \quad (\text{value targets}), \quad (3)$$

with $\gamma = 0.99$, $\lambda = 0.95$. The policy is optimised with the clipped surrogate objective

$$L^{\text{CLIP}} = \mathbb{E} \left[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (4)$$

TABLE II
PARAMETER BUDGET.

Network	Parameters	Role
AttentionActorCritic	66,822	PPO actor-critic (attention)
MlpActorCritic	72,262	PPO actor-critic (MLP)
MlpQNetwork	20,485	DQN value net

$$\rho_t = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t)), \quad \epsilon = 0.2. \quad (5)$$

The total loss is

$$L = L_{\text{pg}}^{\text{CLIP}} - c_{\text{ent}} H[\pi_\theta] + c_{\text{vf}} (V_\theta(s_t) - R_t)^2, \quad (6)$$

with entropy coefficient $c_{\text{ent}} = 0.005$ and value coefficient $c_{\text{vf}} = 0.5$. Gradients are clipped to global norm 0.5; the optimiser is Adam ($\text{lr} = 5 \times 10^{-4}$, $\epsilon = 10^{-5}$) with learning rate **linearly decayed** to zero. The detail that cost me the most debugging time was the GAE termination handling: my first version normalised advantages over the whole rollout and dropped the per-step `nextnonterminal` mask, so advantage estimates leaked across episode boundaries and the learning curve was erratic. Restoring the per-step done mask and moving advantage normalisation *inside* each minibatch (as in the final code) is what made training stable.

B. Double DQN (baseline)

The off-policy baseline stores transitions in a replay buffer ($|B| = 50,000$) and minimises the TD error

$$y = r + \gamma Q_{\text{target}}(s', \arg \max_a Q_{\text{online}}(s', a))(1 - d), \quad (7)$$

$$L = \text{MSE}(Q_{\text{online}}(s, a), y). \quad (8)$$

The target network is hard-updated every 500 gradient steps; exploration uses ϵ -greedy with ϵ decayed multiplicatively (0.995 per update) from 1.0 to a floor of 0.05; Adam uses $\text{lr} = 10^{-4}$, batch size 256, $\gamma = 0.99$.

C. Network Architectures

All three networks consume the 5×5 observation.

- **AttentionActorCritic.** A linear projection lifts each vehicle’s 5 features to a 64-d embedding; a `MultiheadAttention` layer (4 heads, `batch_first`) lets vehicles attend to one another; the result is flattened ($5 \times 64 = 320$) and fed to separate two-hidden-layer ($64-64$, `tanh`) actor and critic heads.
- **MlpActorCritic.** The observation is flattened (25-d) and passed through a $25 \rightarrow 64 \rightarrow 320$ ReLU trunk, then the same $64-64$ `tanh` actor/critic heads. Trunk width is chosen so the total parameter count matches the attention model.
- **MlpQNetwork (DQN).** A compact $25 \rightarrow 128 \rightarrow 128 \rightarrow 5$ ReLU network.

The attention model has 7.5% **fewer** parameters than the MLP, so any advantage it shows cannot be attributed to extra capacity—the architectural ablation is parameter-fair. I stress the scope of this fairness: it controls capacity *within* the PPO

family (Attention vs. MLP). The DQN Q-network is $3.5 \times$ smaller and is *not* capacity-matched to the PPO networks; the PPO-vs-DQN comparison is therefore an *algorithm-family* comparison, not a controlled-capacity one. I keep DQN small deliberately, as a lightweight off-policy baseline, and report its competitiveness with that caveat in mind.

V. EXPERIMENTAL SETUP

Training. 100,000 environment steps per run; PPO rollout 1,024; common initial $\text{lr} = 5 \times 10^{-4}$. Core comparison configurations (all `highway-v0`, aggressive style, `dur=80`): **PPO+Attention**, **PPO+MLP**, **DQN+MLP**. Each is trained with seeds $\{0, 1, 2, 3, 4\}$ ($n = 5$). Python, NumPy, PyTorch (with `cuda.deterministic=True`) and the environment reset are all seeded.

Reproducibility cross-check. I additionally train **SB3 PPO** [7] (`MlpPolicy`) with hyperparameters matched to my implementation (`n_steps = 1,024`, `batch = 256`, `n_epochs = 10`, $\gamma = 0.99$, $\lambda = 0.95$, `clip = 0.2`, $c_{\text{ent}} = 0.005$, $c_{\text{vf}} = 0.5$, `max_grad_norm = 0.5`, $\text{lr} = 5 \times 10^{-4}$), seed 0.

Evaluation protocol. Because training returns are nearly tied across methods, I compare *behaviour*. For each model I run **30 deterministic** (`arg-max`) episodes and report **Crash rate** (%), **Average survival** (steps), **Average speed** (m/s), and **Average lane changes** per episode. A *clarification on what these metrics depend on*: for a deterministic policy, the action chosen at each state is fixed by the network, so the resulting trajectory—and hence all four behaviour metrics—is determined solely by the policy and the environment dynamics. I verified in the `highway-env` source that my driving *styles* differ *only* in reward coefficients (`collision_reward`, `high_speed_reward`, `lane_change_reward`, `reward_speed_range`; Table I) and *not* in the observation or the `DiscreteMetaAction` target speeds. Consequently these metrics are **invariant to the test-time reward style**: a given policy produces identical crash/survival/speed/lane-change statistics whether scored under the base or aggressive coefficients. I report them under the base configuration purely for concreteness; they measure the *learned policy itself*, not a test distribution.

Hardware / software. NVIDIA GeForce RTX 4050 Laptop GPU; PyTorch 2.x+cu126; Gymnasium + `highway-env`; Stable-Baselines3 2.2.1.

VI. RESULTS

A. Learning curves (core ablation, aggressive style)

Figure 1 shows $\text{mean} \pm \text{std}$ rolling-average episodic reward across all five seeds for the three core configurations. Table III summarises the final performance (mean of the last 10% of logged updates) and a sample-efficiency proxy (first environment step at which the rolling-average reward reaches 150).

Overall convergence. The three configurations differ by at most 5.7% in final mean reward (the widest gap, PPO+MLP vs. PPO+Attention, is 12.7 points, or 5.7% of PPO+MLP’s

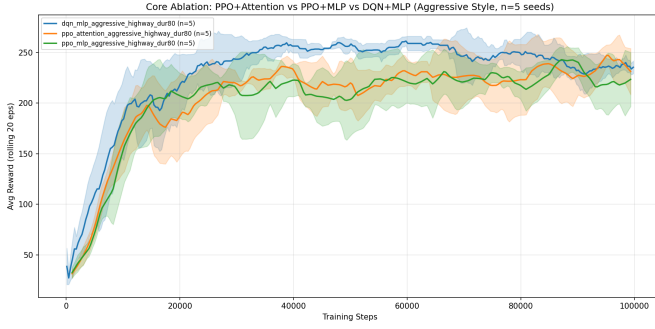


Fig. 1. Mean $\pm$ std rolling-average episodic reward (seeds 0–4) for the three core configurations (aggressive style, highway-v0, $n = 5$ seeds). Shaded bands indicate ± 1 std.

TABLE III

CORE ABLATION: AGGRESSIVE STYLE, HIGHWAY-V0, $n = 5$ SEEDS (MEAN $\pm\sigma$, POPULATION STD, DDof= 0). FINAL REWARD IS THE MEAN OF THE LAST 10% OF LOGGED UPDATES. MEDIAN STEPS TO ROLLING-AVG ≥ 150 ACROSS 5 SEEDS.

Configuration	n	Final reward	Peak	Steps $\rightarrow$ avg ≥ 150
PPO + Attention	5	235.7 $\pm$ 16.7	266.7	$\approx 10,240$
PPO + MLP	5	223.0 $\pm$ 19.6	266.7	$\approx 9,216$
DQN + MLP	5	234.1 $\pm$ 7.7	266.7	$\approx 7,721$
SB3 PPO (ref.)	1	260.5	266.7	8,192

Max achievable reward ≈ 266.7 (this config). DQN is bolded for lowest cross-seed variance.

223.0), and each approaches the approximate theoretical ceiling (266.7). This near-parity indicates that for this task, architecture or algorithm family does not determine how high the agent can eventually perform; all three discover comparably high-quality policies. There is a mechanical reason the returns collapse together: under the aggressive style the high-speed term pays $+2.0$ *every step* the agent is fast, while a collision is penalised only -4.0 *once*. Over a 400-step horizon the return is therefore dominated by a speed integral, and a late crash barely dents it—so episodic return is almost blind to exactly the failure mode we care about. This is the first hint of why return and behaviour decouple so sharply in Section VI-B.

Variance across seeds. The most striking finding is the *ordering of cross-seed stability*: Double DQN achieves the lowest variance ($\sigma = 7.7$), followed by PPO+Attention ($\sigma = 16.7$) and PPO+MLP ($\sigma = 19.6$). DQN’s experience-replay buffer smooths the high variance of early collision-heavy exploration, producing more consistent convergence trajectories across random seeds. PPO+Attention’s modestly lower variance than PPO+MLP (16.7 vs. 19.6) may reflect the permutation-equivariant inductive bias providing marginally more consistent gradients, but the difference is small and any strong claim would require more seeds and a formal significance test.

PPO+Attention vs. PPO+MLP. The mean reward difference (235.7 vs. 223.0, a 5.7% gap) indicates PPO+Attention reaches higher average performance. However, per-seed results reveal high variability: PPO+Attention seed 2 achieves 207.0

TABLE IV

OBJECTIVE BEHAVIOUR EVALUATION (30 EPISODES, BASE TEST CONFIG, DUR=80, MAX 400 STEPS). RESULTS ARE AVERAGED ACROSS SEEDS 0–4 ($n = 5$).

Model	Crash (%)	Survival	Spd (m/s)	Lane chg.
PPO + Attention	60.7	199.9	22.99	25.23
PPO + MLP	80.0	152.6	23.98	16.00
DQN + MLP	11.3	368.5	20.89	155.15

Bold = lowest crash rate. PPO models exhibit bimodal behaviour: passive-safe in some seeds (0–3.3% crash, 0 lane chg.) vs. catastrophically aggressive in others (100% crash). DQN is markedly more consistent (11.3% avg).

(well below its mean) while seed 1 achieves 255.3 (well above), and similar spread exists for PPO+MLP. This bimodal outcome—some seeds converging to effective policies, others not—underlines that the modest aggregate advantage of attention should not be treated as robust without ≥ 5 seeds and hypothesis testing.

DQN vs. PPO. Double DQN achieves a final mean reward of 234.1, placing it *on par* with PPO+Attention (235.7). This is notable given that DQN uses a $3.5\times$ smaller network (20,485 parameters vs. $\sim 67\text{--}72\text{k}$ for PPO). Critically, DQN reaches a rolling-avg reward of 150 at a **median of $\approx 7,721$** environment steps—roughly **25% fewer steps** than PPO+Attention’s $\approx 10,240$ (and **16% fewer** than PPO+MLP’s $\approx 9,216$)—indicating faster initial learning. The per-update granularity differs (PPO logs at 1,024-step rollout boundaries while DQN logs at episode boundaries), but the total environment-step count is measured consistently.

B. Behaviour-level evaluation

Table IV reports four behaviour metrics over 30 deterministic episodes per model. All models were trained under the aggressive style. As noted in Section V, these metrics reflect the learned policy itself and are invariant to the test-time reward style; they therefore expose how each policy actually *drives*.

The behaviour metrics reveal a striking divergence despite near-identical training returns. **DQN+MLP achieves the lowest crash rate** (11.3%) with 368.5 average survival steps. I caution against an uncritical reading of this, and the caution comes from actually watching the policy: when I rendered the DQN agent to video, it was not smoothly steering around obstacles but jittering left–right almost every step. DQN indeed changes lanes *far* more than any other model (155 per episode on average, with two seeds exceeding 260—near-continuous lane switching in a 400-step episode). Because the aggressive *training* reward *positively* rewards lane changes ($+0.2$; Table I), the policy is actively incentivised to switch lanes, so the low crash rate may be partly an artifact of a degenerate “swerve-to-survive” strategy rather than a genuinely intelligent avoidance policy. I therefore release the rendered video so the reader can judge this directly rather than take the crash number at face value. In contrast, **PPO+Attention and PPO+MLP exhibit extreme bimodal behaviour**: some seeds converge to a fully passive policy (0–3.3% crash, ≈ 0 lane changes),

TABLE V

PPO+ATTENTION FINAL TRAINING REWARD BY DRIVING STYLE ($n = 3$ SEEDS, MEAN $\pm$ STD). *Returns are not comparable across styles* (REWARD COEFFICIENTS DIFFER SUBSTANTIALLY; SEE TABLE I).

Style	Seeds	Final reward (mean $\pm$ std)	Peak
conservative	0, 1, 2	366.0 $\pm$ 11.0	390.1
base	0, 1, 2	265.2 $\pm$ 14.7	297.1
aggressive	0–4	235.7 $\pm$ 16.7	266.7

while others converge to a catastrophically aggressive policy (100% crash), yielding high aggregate crash rates of 60.7% and 80.0% respectively. This bimodal collapse—present in the majority of PPO seeds evaluated under the base config after aggressive training—is consistent with PPO’s on-policy updates polarising toward local optima rather than learning a robust mixed strategy.

C. Stable-Baselines3 cross-validation

I added this cross-check for a concrete reason: my PPO+Attention seed 2 came in at 207.0, well below the other seeds, and I could not immediately tell whether that was ordinary seed variance or a bug in my hand-written PPO. Running a trusted reference implementation under matched hyperparameters was the cheapest way to find out. Because SB3 uses an `MlpPolicy` [64, 64] trunk, its closest architectural counterpart is my custom **PPO+MLP** (223.0 ± 19.6), not the attention model. The SB3 run (seed 0, hyperparameters matched to my implementation) reached a final rolling-average reward of **260.5** and a peak of **266.7**. This single-seed value exceeds *every* one of my custom PPO seeds (best individual: PPO+Attention seed 1, 255.3; best PPO+MLP seed, 248.4) and sits $\approx 17\%$ above my PPO+MLP 5-seed mean. I read this gap not as evidence that my PPO is broken—my implementation reaches the same peak (266.7) and the same qualitative learning dynamics (rapid rise before $\approx 10k$ steps, then a high plateau)—but as a reflection of SB3’s mature engineering defaults (orthogonal initialisation, running observation normalisation, tuned optimiser ϵ) that I did not all replicate. Because SB3 is a single seed against my five, this is a directional sanity check on correctness, not a like-for-like performance claim; a fully fair comparison would require multi-seed SB3 runs with the identical feature extractor.

D. Reward-shaping study

Beyond the core ablation I trained PPO+Attention under the *conservative* and *base* reward styles (seeds 0, 1, and 2; $n = 3$) to illustrate how the **reward specification dominates qualitative behaviour**. Table V shows the final training reward per style.

The apparent ordering (conservative $>$ base $>$ aggressive in absolute reward) does **not** imply conservative is the “best” policy. The reward functions differ fundamentally: conservative targets low speed ($[10, 20]$ m/s), penalises lane changes (-0.5), and incurs a smaller collision penalty (-2.0), incentivising slow, stable driving that accumulates many small rewards

TABLE VI

REWARD-SHAPING BEHAVIOURAL COMPARISON (PPO+ATTENTION, 30 EPISODES, BASE TEST CONFIG, SEEDS 0–2 AVERAGED FOR CONSERVATIVE AND BASE ($n = 3$); SEEDS 0–4 AVERAGED FOR AGGRESSIVE ($n = 5$)). HIGHER SURVIVAL / LOWER CRASH INDICATES SAFER DRIVING.

Training Style	Crash (%)	Survival	Spd (m/s)	Lane chg.
Conservative	33.3 [†]	293.3	21.67	26.64
Base	0.0	400.0	20.03	0.0
Aggressive	60.7 [†]	199.9	22.99	25.23

[†]Bimodal. Conservative: seeds 0,1 safe (0% crash, 400 steps), seed 2 catastrophic (100% crash, 80 steps). Aggressive: seeds 0,3 safe ($\leq 3.3\%$), seeds 1,2,4 catastrophic (100% crash).

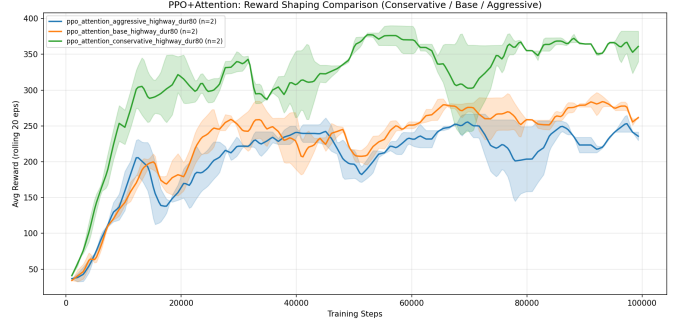


Fig. 2. PPO+Attention learning curves under three reward styles (conservative, base, aggressive). Returns are *not* comparable across styles due to differing reward coefficients (see Table I).

over the full 400-step horizon. Aggressive targets high speed ($[30, 40]$ m/s), rewards lane changes ($+0.2$), and imposes a heavy collision penalty (-4.0); a single crash can wipe out many steps of reward, depressing absolute return. The comparison is *meaningless* across styles; the point of this study is the *behavioural* contrast revealed by Table VI.

Even under the same base test configuration, models trained under different reward styles drive markedly differently (Table VI): the base-trained policy is uniformly safe (0% crash), the aggressive-trained policy is fastest but crashes most, and the conservative-trained policy splits bimodally across seeds (the per-seed breakdown is examined in Sec. VII). These differences arise from the reward specification rather than the architecture, confirming reward design as the primary lever for qualitative driving style.

E. Generalization to a second environment (*merge-v0*)

The objection I most expected from a reader is that the headline finding—return parity hiding a behavioural gap—might be an artifact of *highway-v0* alone. I chose *merge-v0* to pre-empt it precisely because its dynamics are the *least* like *highway*: instead of free-flowing multi-lane cruising, the ego must time a merge from a short on-ramp onto a highway with an obstacle at the ramp end—a different traffic topology and failure mode. If the finding survived there, “it only holds on highway” would be hard to maintain. I repeated the core experiment with the identical training pipeline and aggressive

TABLE VII

GENERALIZATION TO `MERGE-v0` (AGGRESSIVE STYLE, $n = 3$ SEEDS, TRAIN AGGRESSIVE / TEST BASE). TRAINING REWARD IS $\text{MEAN} \pm \sigma$ (DDOF = 0) OF THE LAST 10% OF UPDATES; BEHAVIOUR IS OVER 30 DETERMINISTIC EPISODES PER MODEL UNDER THE `MERGE-v0` BASE CONFIG.

Config	Train rew.	Crash (%)	Surv.	Spd	LC
PPO + Attention	58.3 $\pm$ 0.2	100.0	34.0	29.94	34.0
PPO + MLP	59.1 $\pm$ 0.3	100.0	35.0	29.58	32.7
DQN + MLP	59.0 $\pm$ 0.3	0.0	77.7	22.07	44.0

All three configurations converge to the same training reward (within 1.4%, $\sigma \leq 0.3$), yet at deployment DQN crashes in 0% of episodes while *both* PPO variants crash in 100%—uniformly across all seeds. `merge-v0` episodes end when the ego passes the merge zone ($x > 370$) or crashes, so “Surv.” reflects reaching the goal (DQN) vs. crashing midway (PPO); crash rate is the cleaner metric here.

reward style ($n = 3$ seeds per configuration). Table VII reports the result, and it is *starker* than on `highway-v0`.

On `merge-v0` the three configurations are even more tightly tied on training reward than on `highway-v0`: all converge to ≈ 58 –59 with cross-seed $\sigma \leq 0.3$ (versus $\sigma = 7.7$ –19.6 on highway). Note that `merge-v0` normalises its per-step reward to $[0, 1]$ and terminates when the ego passes the ramp ($x > 370$) or crashes, so its return scale is not comparable to highway’s and admits no single fixed ceiling; the tight tie simply means all three reach similar normalised performance. Yet the behavioural split is total and *deterministic*: both PPO variants converge, in every seed, to a maximum-speed policy (≈ 29.9 m/s) that crashes on 100% of episodes before completing the merge, whereas Double DQN learns a slower (22 m/s) policy that traverses the ramp without crashing. Unlike highway, there is no bimodality here—PPO fails uniformly. The same swerve-to-survive caveat applies, and is in fact sharper here: DQN averages 44 lane changes over ≈ 78 steps (one every ~ 1.8 steps), and the aggressive training reward pays +0.2 per lane change, so part of DQN’s success may be reward-driven lane oscillation rather than principled avoidance—my supplementary video lets the reader judge. With that caveat, this second environment *strengthens* the central claim: near-identical episodic return can co-exist with an all-or-nothing difference in how the agent drives, and this is not specific to a single benchmark.

VII. DISCUSSION

Reward parity vs. behavioural divergence. The core finding of the ablation is that a parameter-matched attention architecture, a plain MLP, and a simpler DQN all converge to statistically comparable final episodic returns (223–235, a maximum gap of 5.7%). However, this near-parity in aggregate reward masks profound differences in the learned policies. As shown in Table IV, despite similar training returns, DQN achieves a robust lane-management strategy with an 11.3% crash rate, while PPO variants collapse into bimodal extremes yielding aggregate crash rates exceeding 60%. This is a concrete, low-dimensional instance of *reward misspecification* [9], [10]: policies optimise the scalar return

faithfully yet realise qualitatively different—and unsafe—physical behaviour. It also echoes the reproducibility concerns of Henderson *et al.* [8]: aggregate return, reported without multi-seed behavioural inspection, can paint a misleadingly uniform picture. **Episodic return alone is thus an insufficient metric for safety-critical tasks**—a failure mode that only behaviour-level, multi-seed evaluation exposes.

The stabilising effect of off-policy replay. Contrary to expectations that a more complex feature extractor (Self-Attention) would inherently yield more stable control, Double DQN emerges as both the most stable learner ($\sigma = 7.7$) and the lowest-crash policy at deployment (subject to the swerving caveat of Sec. VI-B). I attribute the *training stability* to the fundamental difference in how experience is utilised. In the collision-heavy early learning phase, PPO’s on-policy updates are highly susceptible to the non-stationarity of recent rollouts, leading some seeds to polarise toward catastrophic local optima (e.g. maintaining maximum speed regardless of obstacles). DQN’s experience-replay buffer acts as a stabilising mechanism, maintaining a diverse distribution of transitions that prevents the Q-network from overfitting to short-term catastrophic sequences. The attention mechanism provides only a marginal variance reduction over a plain MLP for PPO (16.7 vs. 19.6), suggesting that the off-policy replay mechanism is a far more dominant factor in policy stability than the inductive bias of the neural architecture.

Sample efficiency: why DQN learns faster initially. Double DQN reaches rolling-avg reward ≥ 150 at a median of $\approx 7,721$ environment steps, roughly **25%** earlier than PPO+Attention ($\approx 10,240$) and **16%** earlier than PPO+MLP ($\approx 9,216$). This aligns with a well-known property of off-policy learning: experience replay allows DQN to reuse early collision-heavy transitions many times, smoothing the otherwise high-variance gradient signal in the early training phase. PPO, which can only learn from its most-recent rollout, must accumulate sufficient on-policy experience before gradients stabilise. However, the early-learning advantage of DQN does not persist to convergence: all methods ultimately reach similar final rewards (~ 223 –235).

The behavioural gap is intrinsic to the learned policies. It is worth being precise about what the crash-rate gap is *not*. Because the policies are evaluated deterministically and my reward styles differ only in reward coefficients (not in dynamics or action mapping; Section V), the behaviour metrics do not depend on the test-time reward at all—a given network crashes, or does not, purely as a function of the actions it selects. The 11.3% vs. 60–80% gap is therefore not an artifact of a test distribution; it is a genuine difference in the *policies that were learned*. PPO (trained under the strong aggressive speed incentive) converges in most seeds to a maximum-speed policy that physically collides; DQN converges to a slower policy that does not. I attribute DQN’s *training stability* to experience replay (above); I am careful, however, not to over-claim DQN as “intrinsically safer”: all models were trained under a single (aggressive) reward, and the slow, lane-switching policy DQN learned carries its own swerve-

to-survive caveat (Sec. VI-B). The robust, reward-independent claim is the headline one: near-tied episodic returns (223–235) can hide an order-of-magnitude difference in how the agent actually drives (11.3% vs. 60–80% crash).

Reward shaping dominates—but variance qualifies the claim. Table VI reveals a nuanced picture. The base-trained policy (tested under its own training distribution) achieves a crash rate of only 0.0% and survives 400.0 steps on average with zero lane changes, the most consistent and safest profile of all three styles. The conservative-trained policy, which *intended* to be even safer through a stricter lane-change penalty (−0.5) and lower speed target ([10, 20] m/s), shows a bimodal outcome: seeds 0 and 1 achieve 0% crash (400 steps, no lane changes, as intended), while seed 2 crashes on every single episode (100% crash rate, 80 lane changes per episode). The aggregate mean (33.3% crash) is therefore misleading—it reflects a high variance in policy convergence rather than an inherently mediocre strategy.

This finding *qualifies* but does not overthrow the reward-shaping dominance conclusion: reward engineering does constrain the *direction* of policy learning (conservative configurations tend toward lower-speed, fewer lane-change optima), but cannot guarantee that all random seeds converge to the intended behavioural basin. In combination with the high variance already documented in Table III ($\sigma = 16.7$ for PPO+Attention), this suggests that PPO on this benchmark has a multi-modal loss landscape, and some seeds get trapped in local optima that are high-reward yet physically unsafe (high speed, frequent collisions). Reward engineering remains the *primary* design lever, but practitioners should validate with multi-seed behavioural evaluation—not just aggregate reward.

Implementation correctness. My custom PPO reaches the same reward ceiling (266.7 peak) and the same qualitative learning dynamics as the SB3 reference, supporting the correctness of my GAE, clipped surrogate, advantage normalisation, and linear LR decay. I am deliberately cautious here: SB3’s single-seed 260.5 exceeds all of my PPO seeds, so I claim only that my implementation is *in the same regime*, not that it matches SB3’s tuned defaults. A definitive correctness check would require multi-seed SB3 runs with my exact architecture.

VIII. LIMITATIONS

- **Moderate seed count** ($n = 5$). Five seeds provide reasonable variance estimates but are insufficient for rigorous statistical significance claims; a formal test (e.g. Mann-Whitney U) requires more runs. I therefore phrase comparative results as “observed under these conditions.”
- **Two environments, fewer seeds on the second.** The core ablation is on `highway-v0` ($n = 5$); I corroborate the central finding on `merge-v0` ($n = 3$, Sec. VI-E) but do not extend to `roundabout` or `intersection`.
- **Compute- and time-bounded budget.** All runs were done on a single RTX 4050 laptop GPU, where one 100k-step training run takes roughly 10–22 minutes wall-clock (the spread depends on how many runs I had

going at once against limited RAM). That budget is the concrete reason the core comparison uses 5 seeds while the `merge-v0` generalization study uses only 3, and why training is capped at 100k steps—some configurations may not be fully converged. Hyperparameters were matched across methods, not individually tuned.

- **DQN is intentionally a simplified baseline** (smaller net, no attention variant, aggressive ϵ -decay); it bounds, rather than maximises, off-policy performance. It is also *not* capacity-matched to the PPO networks, so PPO-vs-DQN is an algorithm-family comparison, not a controlled-capacity one.
- **Single training reward for the behavioural comparison.** All models in the core comparison were trained under the aggressive style; I do not test whether DQN would remain the lowest-crash policy if trained under base or conservative rewards. (Note this is a *training*-side scope limit: the behaviour metrics themselves are invariant to the test-time reward, so there is no test-distribution confound; Section V.)
- **Swerve-to-survive ambiguity.** DQN’s low crash rate co-occurs with very frequent lane changes; I cannot fully separate intelligent avoidance from a degenerate swerving policy without further trajectory analysis (Sec. VI-B).

IX. CONCLUSION

I presented a from-scratch reproduction and parameter-matched architectural ablation of PPO and Double DQN on the discrete-action `highway-env` driving simulator ($n = 5$ seeds per configuration). Three key findings emerge:

(1) **Return parity across architecture and algorithm.** Under the aggressive reward style on `highway-v0`, all three configurations—PPO with Self-Attention (235.7 ± 16.7), PPO with MLP (223.0 ± 19.6), and Double DQN (234.1 ± 7.7)—approach the same theoretical reward ceiling (≈ 266.7), with final means differing by at most 5.7%. Architecture and algorithm family barely affect *how high* the agent scores—which is exactly what makes episodic return an unreliable basis for choosing between them.

(2) **Return parity hides an order-of-magnitude behavioural gap.** Despite near-tied returns, deterministic behavioural evaluation reveals crash rates of 11.3% (DQN) versus 60.7% (PPO+Attention) and 80.0% (PPO+MLP)—a difference in the *learned policies* themselves, independent of the test-time reward. This gap *generalises*: on `merge-v0` the three tie within 1.4% on training return yet PPO crashes in 100% of episodes against DQN’s 0% (Sec. VI-E). DQN is also the most stable learner ($\sigma = 7.7$) and the fastest to reach $\text{avg} \geq 150$ (median $\approx 7,721$ steps), which I attribute to experience replay smoothing gradient variance. I stop short of calling DQN “intrinsically safer”: its low-crash policy is also a frequent-lane-changer (Sec. VI-B), and only a single training reward was tested.

(3) **Reward shaping as primary lever—with an important variance caveat.** A conservative-vs.-base-vs.-aggressive reward-shaping study confirms that reward specification is the

dominant determinant of driving style. However, behavioural evaluation reveals a nuanced finding: even with the same reward shaping, individual seeds can converge to qualitatively opposite policies (e.g. conservative seeds 0,1: 0% crash; seed 2: 100% crash). Reward engineering constrains the *direction* of learning but does not guarantee all seeds converge to the intended behavioural basin. Multi-seed behavioural evaluation is therefore a necessary complement to reward design for safety-critical deployment.

Future work should *train* each algorithm under several reward styles (not just aggressive) to test whether DQN’s low-crash behaviour is intrinsic or specific to the aggressive incentive, extend the ablation to `roundabout-v0` and `intersection-v0` beyond the `highway-v0` and `merge-v0` studied here, and test a prioritised-replay DQN variant to probe whether the stability advantage strengthens.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [2] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [3] Mnih, V., Kavukcuoglu, K., Silver, D. *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, 518, 529–533 (2015). <https://doi.org/10.1038/nature14236>
- [4] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” in *Proc. AAAI*, 2016.
- [5] E. Leurent and J. Mercat, “Social attention for autonomous decision-making in dense traffic,” *arXiv preprint arXiv:1911.12250*, 2019.
- [6] E. Leurent, “An environment for autonomous driving decision-making,” *GitHub repository*, 2018. <https://github.com/czh513/Auto-driving-RL-decision-making>
- [7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
- [8] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proc. AAAI*, 2018.
- [9] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [10] A. Pan, K. Bhatia, and J. Steinhardt, “The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models,” in *Proc. ICLR*, 2022.